

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
```

```
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)
```

```
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets
```

```
y = y.rename(columns={y.columns[0]: "target"})
```

```
y["target"] = y["target"].map({"M": 1, "B": 0})
```

```
from sklearn.model_selection import train_test_split
y = y["target"].values.ravel()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression(max_iter=5000)
log_reg.fit(X_train, y_train)
```

```
y_pred = log_reg.predict(X_test)
y_prob = log_reg.predict_proba(X_test)[:, 1]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
```

Accuracy: 0.9440559440559441

Precision: 0.9787234042553191
Recall: 0.8679245283018868
ROC AUC: 0.9949685534591195

```
pd.set_option('display.max_rows', None)
```

```
coef = pd.DataFrame({  
    "feature": X.columns,  
    "coef": log_reg.coef_[0]  
}).sort_values(by="coef", ascending=False)  
  
print(coef)
```

	feature	coef
26	concavity3	1.052987
28	symmetry3	0.676446
25	compactness3	0.546146
21	texture3	0.515671
6	concavity1	0.453649
27	concave_points3	0.443274
2	perimeter1	0.279879
24	smoothness3	0.261362
7	concave_points1	0.257119
5	compactness1	0.217764
8	symmetry1	0.204251
12	perimeter2	0.176273
4	smoothness1	0.157180
13	area2	0.098706
10	radius2	0.088632
29	fractal_dimension3	0.070641
22	perimeter3	0.067850
18	symmetry2	0.042789
16	concavity2	0.033950
9	fractal_dimension1	0.032495
17	concave_points2	0.030874
14	smoothness2	0.020027
23	area3	0.015713
19	fractal_dimension2	-0.010875
3	area1	-0.023619
15	compactness2	-0.048653
20	radius3	-0.149610
1	texture1	-0.214348

```
0          radius1 -0.999944
11         texture2 -1.347362
```

```
top3_features = [
    #considerando o modulo mesmo
    "concavity3",
    "radius1",
    "texture2"
]
X_reduced = X[top3_features]
```

```
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reduced, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
```

```
log_small = LogisticRegression(max_iter=5000)
log_small.fit(X_train_r, y_train_r)
```

```
y_pred_r = log_small.predict(X_test_r)
y_prob_r = log_small.predict_proba(X_test_r)[:, 1]
```

```
print(f"Completo ({X_test.shape[1]} variáveis)")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
print("\n", "=" * 20, "\n")
print(f"REDUZIDO ({X_test_r.shape[1]} variáveis)")
print("Accuracy:", accuracy_score(y_test_r, y_pred_r))
print("Precision:", precision_score(y_test_r, y_pred_r))
print("Recall:", recall_score(y_test_r, y_pred_r))
print("ROC AUC:", roc_auc_score(y_test_r, y_prob_r))
```

```
Completo (30 variáveis)
Accuracy: 0.9440559440559441
Precision: 0.9787234042553191
Recall: 0.8679245283018868
ROC AUC: 0.9949685534591195
```

=====

REDUZIDO (3 variáveis)
Accuracy: 0.9090909090909091
Precision: 0.9347826086956522
Recall: 0.8113207547169812
ROC AUC: 0.980083857442348